

Nodlatch Build Playbook

How the demo works end to end on your laptop and iQOO phone, which files exist, and the order to build them in during the 26–27 September event.

Nothing here is code; code starts when the clock starts.

The three programs

Two run on the laptop, one on the phone. The agent is never allowed to write into the real repository; it only ever writes into a scratch copy. Only the executor writes to the real repo, and only after the phone has signed a decision.

LAPTOP

AI coding agent

Claude Code (`claude - p`), Antigravity, or a local model via Ollama. Gets the task and a scratch copy of the repo. Untrusted.

LAPTOP

Nodlatch executor

Small Python CLI you write at the event: `propose` , `execute` . Builds proposals, verifies decisions, applies, tests, rolls back.

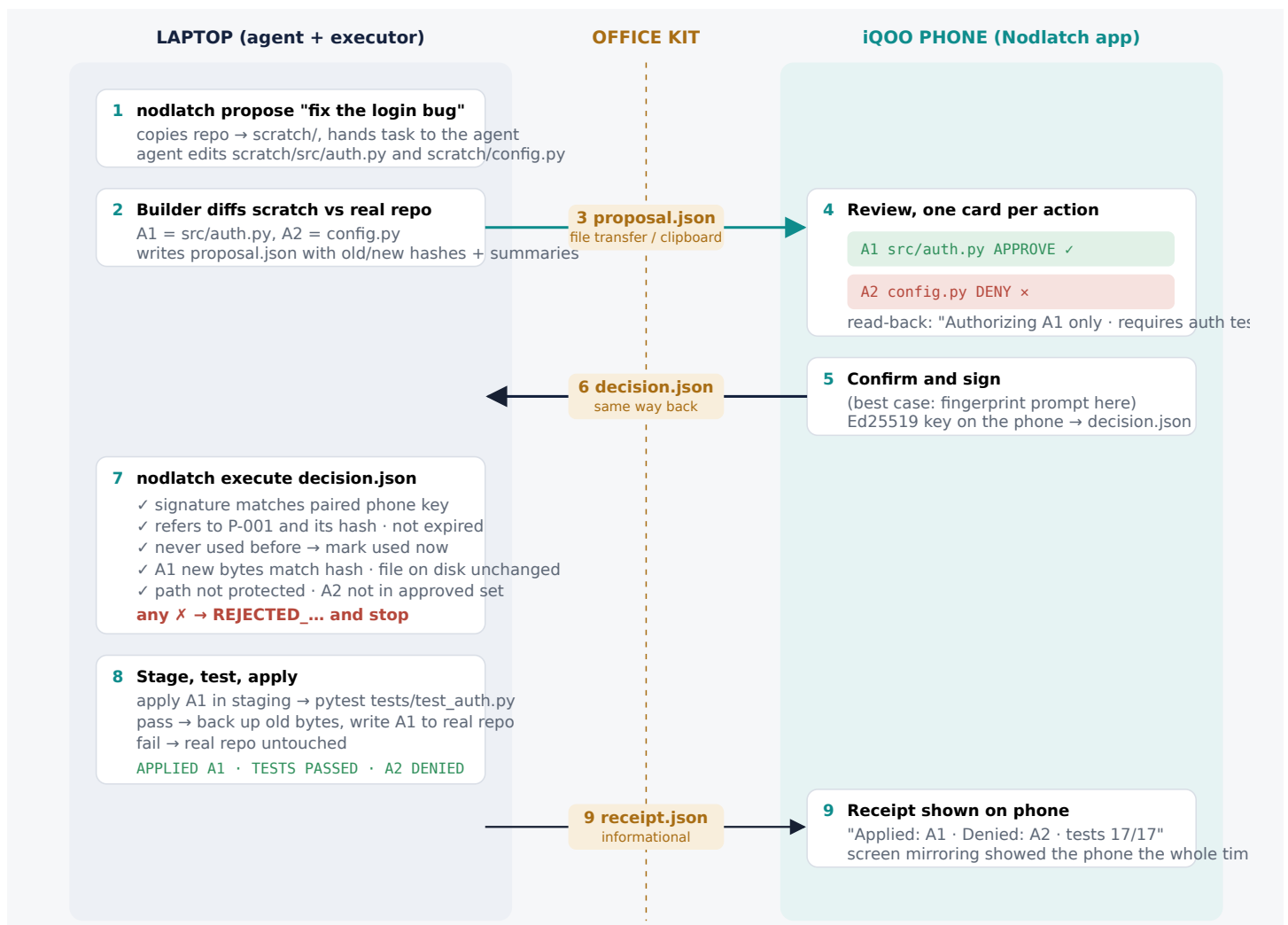
IQOO PHONE

Nodlatch app

Shows the proposal, one card per action, approve/deny, read-back, signs the decision with a key that never leaves the phone.

Office Kit is not a program you write. It is the event-provided bridge (file transfer, shared clipboard, screen mirroring) that moves three small JSON files between the two devices. The executor never trusts a file because it came through Office Kit; it re-checks everything.

Best case, start to finish



The whole demo is nine steps and three files. Steps 1, 2, 7, 8 are the laptop; 4, 5, 9 are the phone; 3, 6, 9 are Office Kit carrying a JSON file across. The laptop never applies anything the phone did not sign, and the phone never touches the repository.

"Best case" hides three unknowns you cannot resolve until you hold the loaner phone: whether Office Kit file transfer and clipboard behave the way steps 3 and 6 need, whether the phone can do Ed25519 signing and a fingerprint prompt easily, and whether the executor may run on the laptop during a Red Light evaluation. Test all three in hour one.

The three files

Keep them small, single-line-safe (base64 the content blobs if they go through the clipboard), and keep the exchange folder *outside* the git repo.

```
// proposal.json (laptop → phone)
{
  "proposal_id": "P-001",
  "repo": "demo-repo",
  "created_at": "2026-09-26T12:40:00+05:30",
```

```

"expires_at": "2026-09-26T12:50:00+05:30",
"actions": [
  { "action_id": "A1", "type": "modify_file", "path": "src/auth.py",
    "expected_old_sha256": "9f2c...", "new_file_sha256": "b71e...",
    "summary": "Fix token expiry comparison in verify_token()", "risk": "low" },
  { "action_id": "A2", "type": "modify_file", "path": "config.py",
    "expected_old_sha256": "0a44...", "new_file_sha256": "c3d9...",
    "summary": "Change SESSION_TTL from 3600 to 86400", "risk": "medium" }
],
"test_suites": [ { "id": "auth-suite-v1", "min_tests": 17 } ],
"review_view": { "A1": "...redacted diff text...", "A2": "..." },
"commitment_sha256": "e88a..." // hash of everything above except review_view
}

// decision.json (phone → laptop)
{
  "decision_id": "D-001", "proposal_id": "P-001", "commitment_sha256": "e88a...",
  "approved": ["A1"], "denied": ["A2"], "required_test_suite": "auth-suite-v1",
  "signed_at": "2026-09-26T12:42:10+05:30",
  "signature": "base64-ed25519-over-exact-bytes-of-the-fields-above"
}

// receipt.json (laptop → phone, informational)
{ "decision_id": "D-001", "status": "APPLIED", "applied": ["A1"],
  "denied": ["A2"], "tests": "17 passed", "finished_at": "..." }

```

The new file contents themselves never travel to the phone; they stay in `nodlatch/store/<sha256>` on the laptop. The phone authorizes a hash, and the executor checks the bytes against it.

Executor checks, in this order

Every check that fails prints one status line and stops. Nothing is written before check 6 has marked the decision as used.

- | | | |
|---|--|----------------------------|
| 1 | Parse decision.json strictly (unknown fields → reject) | REJECTED_MALFORMED |
| 2 | Verify Ed25519 signature against the paired phone public key | REJECTED_BAD_SIGNATURE |
| 3 | proposal_id matches a stored proposal and its commitment hash | REJECTED_PROPOSAL_MISMATCH |
| 4 | Laptop clock ≤ expires_at | REJECTED_EXPIRED |
| 5 | decision_id not in used_decisions.json | REJECTED_REPLAY |
| 6 | Write decision_id to used_decisions.json now (single-use, before any mutation) | — |

7	approved + denied IDs all exist; approved set non-empty	REJECTED_ZERO_ACTIONS
8	For each approved action: stored bytes hash == new_file_sha256	REJECTED_ARTIFACT_HASH_MISMATCH
9	For each approved action: current file hash == expected_old_sha256	REJECTED_SOURCE_STATE_MISMATCH
10	Path is relative, no <code>..</code> , no symlink, not in protected list (<code>.env*</code> , <code>*.pem</code> , <code>tests/**</code> , <code>pytest.ini</code> ...)	REJECTED_PROTECTED_TARGET
11	Resolve <code>required_test_suite</code> from the executor's own <code>trusted_suites.json</code> (never from the phone or agent)	REJECTED_UNKNOWN_SUITE
12	Apply approved bytes in a staging copy; run the suite; require exit 0 and $\geq$ min_tests collected	TESTS_FAILED_NOT_APPLIED
13	Re-check step 9 on the real tree, back up old bytes to <code>backups/D-001/</code> , write new bytes, verify hash	EXECUTION_ERROR_ROLLED_BACK
14	Write <code>receipt.json</code>	APPLIED

Build order: seven slices

Evaluation Round 1 is about eight hours after hacking starts, so slice 0 must be demoable first, even though it has no security in it yet. Each slice adds files without rewriting the previous ones. Times are rough and assume three people working in parallel where the columns allow.

Slice 0 • The vertical slice ~2.5 h • no signing, no AI, no checks

LAPTOP FILES

- `demo-repo/src/auth.py`, `demo-repo/config.py`, `demo-repo/tests/test_auth.py` (small, ~17 tests)
- `nodlatch/propose.py` — for now takes a hand-written "new auth.py", writes `exchange/out/proposal.json`
- `nodlatch/execute.py` — reads `exchange/in/decision.json`, copies approved bytes into the repo
- `nodlatch/store/`, `exchange/` (outside the repo)

PHONE FILES

- `app/index.html` — one page: "Import proposal" button, one card per action with Approve/Deny, "Export decision" button
- `app/app.js` — parse proposal, build decision JSON, show it / copy it
- Pick the shell now: PWA in the phone browser, or a minimal Android/Flutter app. Whichever the first-hour test says is faster.

Done when: you tap Approve on the phone and one file changes on the laptop.
Show this to a mentor before touching anything else.

Slice 1 • Exactness ~1.5 h

LAPTOP FILES

- `nodlatch/hashing.py` — sha256 of files and of the proposal commitment
- `propose.py` now records `expected_old_sha256`, `new_file_sha256`, `commitment_sha256`
- `execute.py` adds checks 3, 7, 8, 9 and the status codes

PHONE FILES

- `app.js` shows action IDs, paths, summaries; decision carries `commitment_sha256`
- Read-back screen: "Authorizing A1 only" before Export

Done when: flipping one byte in `store/` after approval makes execute print `REJECTED_ARTIFACT_HASH_MISMATCH`. That is demo 2.

Slice 2 • Default deny ~1 h

LAPTOP FILES

- `nodlatch/policy.py` — protected path list, path normalisation
- `nodlatch/state/used_decisions.json` — checks 4, 5, 6, 10
- Strict JSON parsing (check 1)

PHONE FILES

- `app.js` adds `decision_id`, `signed_at`; shows proposal expiry countdown

Done when: running execute twice on the same decision prints `REJECTED_REPLAY`. That is demo 4.

Slice 3 • Test gate and rollback ~1.5 h

LAPTOP FILES

- `nodlatch/trusted_suites.json` — `{"auth-suite-v1": {"cmd": ["pytest", "tests/test_auth.py", "-q"], "min_tests": 17}}`
- `nodlatch/testgate.py` — staging copy, run suite, parse count, checks 11, 12
- `nodlatch/backups/` — check 13, restore on failure

PHONE FILES

- Suite picker on the read-back screen (one option is fine)
- `receipt` view: status, applied, denied, test count

- A second fixture proposal whose change breaks a test

Done when: the breaking fixture prints `TESTS_FAILED_NOT_APPLIED` and the repo is byte-identical to before. That is demo 3.

Slice 4 • Signing and pairing ~2 h, fingerprint time-boxed to 1 h

LAPTOP FILES

- `nodlatch/verify.py` — Ed25519 verify (PyNaCl or `cryptography`)
- `nodlatch/trusted_keys/phone.pub` — written once at pairing
- `nodlatch pair` — show the phone's public key fingerprint on both screens, human compares, confirm

PHONE FILES

- `app/keys.js` — generate keypair once, store private key on the phone (WebCrypto Ed25519 if the browser has it, else `tweetnacl-js`; Android Keystore if native)
- Sign the exact serialized bytes of the decision fields; never re-serialize on the laptop
- Optional: biometric prompt before signing. If it doesn't work in an hour, ship a confirm tap and say so.

Done when: a decision edited after signing prints `REJECTED_BAD_SIGNATURE`, and an unedited one still applies.

Slice 5 • The real agent ~2 h

LAPTOP FILES

- `nodlatch/agent.py` — copy repo to `scratch/`, run the agent there (Ollama chat call, or `claude -p "..."` with `cwd = scratch`), then diff `scratch` vs `repo` into actions
- `propose.py` now calls `agent.py` instead of the hand-written fixture
- Keep the fixtures for demos 2–4; they must be reproducible on stage

PHONE FILES

- Nothing required. Stretch: typed intent box ("only the auth change") → pre-selects Approve/Deny, human still confirms

Done when: "fix the login bug" produces a real A1/A2 proposal from the model with no hand edits, and demo 1 runs on it.

Slice 6 • Polish and rehearsal whatever is left, minimum 1.5 h

LAPTOP

- `nodlatch/redact.py` — mask API-key/JWT/.env-style values in `review_view`; mark changed secrets and reject them
- Coloured status output; one-command demo script per scenario

PHONE

- Haptic on approve/deny, receipt animation, "SIMULATED EXECUTION — DEMO ONLY" mode for Red Light
- Record the backup video of all four demos

Done when: all four demos run back-to-back in under four minutes, twice in a row, from a clean repo.

The four demos

1 • Selective approval

Real agent proposes A1 + A2. Approve A1, deny A2. Only A1 lands; tests pass; receipt on phone.

APPLIED (A1)

2 • Tampered artifact

Fixture. After approval, flip one byte in `store/`. Executor recomputes the hash.

REJECTED_ARTIFACT_HASH_MISMATCH

3 • Failing test

Fixture whose change breaks `test_auth.py`. Staged, tested, never written.

TESTS_FAILED_NOT_APPLIED

4 • Replay

Run execute again with the decision from demo 1.

REJECTED_REPLAY

Cut list, in order

If you are behind, drop from the top. Never cut slice 0-2.

1. Typed/voice intent interpreter on the phone
2. Biometric prompt (keep a confirm tap)
3. Second transport (keep whichever of file transfer / clipboard worked first)
4. Redaction beyond a single API-key pattern
5. Demo 4 (replay), then demo 2 (tamper). Keep demo 1 and demo 3.
6. Real agent: fall back to the fixture proposal and say so on the slide

Hour-one tests on the loaner phone

TEST	HOW	IF IT FAILS
Office Kit file transfer	Drag a text file laptop → phone and back. Note where it lands on each side.	Use shared clipboard; add Import/Export buttons that paste/copy.
Shared clipboard, both directions	Copy on laptop, paste on phone; copy on phone, paste on laptop. Check line endings survive.	Base64 the payloads; fall back to file transfer.
Ed25519 in the phone browser	Open a page that calls <code>crypto.subtle.generateKey({name:"Ed25519"})</code> .	Load tweetnacl-js, or go native Android Keystore.
HTTPS / secure context for a PWA	Can the phone open your page over https or localhost-equivalent?	Serve the app as a local file-installed PWA, or switch to a native shell.
Biometric prompt	WebAuthn <code>navigator.credentials.create()</code> with platform authenticator, or Android BiometricPrompt.	Confirm tap only; state it on the limitations slide.
Executor during Red Light	Ask an organiser at the 10:00 teach-in.	Phone-only SIMULATED mode for that evaluation.

Who does what

PERSON	OWNS	FIRST TASK AT 11:00
Phone dev	<code>app/</code> , signing, pairing UI, Red Light work	Hour-one tests 1–5, then slice 0 phone page
Executor dev	<code>nodlatch/</code> , demo repo, test gate, fixtures	Demo repo + slice 0 propose/execute
Agent + pitch	<code>agent.py</code> , Ollama/Claude setup, deck, video, organiser questions	Get a model answering locally; ask about Red Light; then help wherever slice 0 is stuck

All three names go on every commit day. Create the repo when the clock starts, commit after every slice, and keep the commit messages honest: the history is your proof of original work.